

Atelier — Démarrer le frontend et le backend

Durée : 15 minutes · Aucun code réseau à écrire en S1

Préparez avant la séance Node.js 24.x, npm, Git, un éditeur et un navigateur. Un accès au registre npm est nécessaire pour les dépendances. Les commandes sont à lancer dans un terminal, une ligne après l'autre.

1. Vérifier les outils — 2 minutes

```
node --version
npm --version
git --version
```

La version de Node doit commencer par `v24`.. Notez les versions si un débogage est nécessaire. Le fichier `.nvmrc` indique également 24 pour les utilisateurs d'un gestionnaire de versions ; installer ce gestionnaire n'est pas demandé.

2. Récupérer le projet — 2 minutes

Après décompression du kit, placez le terminal dans le dossier `sequence-1-v2`, qui contient `messaging-native.bundle` :

```
git clone messaging-native.bundle messaging-websocket
cd messaging-websocket
```

Ce fichier est un dépôt Git local complet. Si votre formateur fournit un dépôt distant, utilisez son URL à la place de `messagerie-native.bundle`.

Vous devez voir `package.json`, `package-lock.json`, `frontend/` et `backend/`. Vous pouvez aussi travailler directement dans le dossier `projet/` livré : cela permet le démarrage, mais ne valide pas le clonage.

3. Installer — 4 minutes

Depuis la racine du projet :

```
npm ci
```

Cette commande installe les dépendances des deux dossiers à partir du verrouillage fourni. Une seule installation suffit. Le premier téléchargement peut dépasser le temps prévu selon votre réseau : signalez-le plutôt que de changer les versions.

4. Lancer les deux services — 4 minutes

Terminal A, à la racine du projet :

```
npm run dev:back
```

Attendez les lignes :

```
Backend prêt : http://127.0.0.1:3000/health  
Point WebSocket préparé : ws://127.0.0.1:3000/chat
```

Cette commande compile le backend puis le démarre. Elle n'active pas de redémarrage automatique : après une future modification du backend, arrêter avec `Ctrl+C` puis relancer la commande.

Terminal B, également à la racine du projet :

```
npm run dev:front
```

Ouvrez `http://127.0.0.1:5173`. La messagerie « Le Salon » doit apparaître. Gardez les deux terminaux ouverts.

5. Vérifier et remplir la checklist — 3 minutes

Dans un autre onglet du navigateur, ouvrez `http://127.0.0.1:3000/health`. Vous devez lire :

```
Backend prêt — séquence 1. Point WebSocket : /chat
```

Adresse	Ce qu'elle vérifie
<code>http://127.0.0.1:5173</code>	Frontend Vue servi par Vite
<code>http://127.0.0.1:3000/health</code>	Backend NestJS démarré, répondant à une requête HTTP
<code>ws://127.0.0.1:3000/chat</code>	Adresse que le code client utilisera en S2 ; ne pas la tester dans la barre d'adresse comme une page Web

Le frontend affiche des messages fictifs. Le bouton « Envoyer (démon) » affiche une explication locale. Le backend est prêt, mais le frontend n'a pas encore son code de connexion au chat.

- ☐ J'ai relevé ma version Node 24.x.
- ☐ J'ai récupéré le bon projet et installé ses dépendances.
- ☐ Le frontend s'affiche sur le port 5173.
- ☐ Le backend répond sur `/health`, port 3000.
- ☐ Je distingue « backend démarré » et « navigateur connecté au chat ».
- ☐ J'ai complété le schéma du cours.

Preuve à déposer : votre schéma et une capture ou un relevé montrant les deux vérifications. En cas de blocage, fournir la commande, le système utilisé et le message d’erreur exact.

Repères dans le projet fourni

Fichier	Rôle ; aucune modification requise aujourd’hui
frontend/src/App.vue	Interface et données fictives
backend/src/main.ts	Démarrage NestJS et activation de WsAdapter
backend/src/chat.gateway.ts	Point WebSocket /chat, sans traitement de message métier
backend/src/health.controller.ts	Réponse HTTP servant au diagnostic d’installation
package.json à la racine	Commandes des deux services

Vite possède sa propre connexion WebSocket pour recharger l’interface en développement. Si vous la voyez dans les outils réseau, elle ne constitue pas une preuve de connexion au chat.

Dépannage

Symptôme	Action
Une commande outil est introuvable	Vérifier l’installation, rouvrir le terminal, demander une aide technique
ENOENT ... package.json	Ouvrir le terminal à la racine du projet cloné
Node incompatible	Basculer sur Node 24.x avant de réinstaller

<code>npm ci</code> échoue sur le réseau	Vérifier accès npm, connexion et proxy de l'établissement
<code>npm ci</code> signale un verrouillage incohérent	Récupérer le kit complet ; conserver le verrouillage fourni
EADDRINUSE ou port 5173 occupé	Arrêter un précédent lancement de ce projet avec <code>Ctrl+C</code> ; demander de l'aide si le port appartient à un autre logiciel
Le front fonctionne mais <code>/health</code> est inaccessible	Examiner le terminal backend et attendre la fin de sa compilation
<code>/health</code> répond mais le front ne s'ouvre pas	Examiner le terminal frontend et vérifier l'URL 5173
<code>/chat</code> ne montre pas de page HTTP	Normal : le point attend une ouverture WebSocket, étudiée en S2
Le bouton n'envoie pas aux autres utilisateurs	Normal en S1 : connexion, messages et diffusion seront ajoutés en S2–S4

Pour terminer, faites `Ctrl+C` dans chaque terminal. Pour reprendre, relancez les deux commandes depuis la racine du projet ; ne réinstallez pas les dépendances à chaque séance.